

Jalon 2

Cadrage & préproduction

Contribution technique au business plan — du MVP en production à la V1

PROJET	Lumia — l'assistant de vie conversationnel qui pense à ce que vous oubliez de demander
LIVRABLE	Jalon 2 — volet « Développement web » (cadrage & préproduction)
FILIÈRE	MBA Développeur Full Stack — Bloc 4 : Conception & déploiement des solutions techniques
AUTEUR	Issa — Lead technique
ÉQUIPE	Antonin (chef de projet & stratégie) · Maëlane (direction artistique) · Issa (lead technique)
MVP EN LIGNE	vocal-assistant-two.vercel.app — en production depuis février 2026
DATE	Juin 2026

Table des matières

1	Cadrage & analyse d'écart V0 → V1	3
2	Spécifications techniques (expression des besoins V1)	5
3	Architecture technique & stack technologique	6
4	Expérience vocale & design system (Liquid Glass + Aurora)	9
5	Infrastructure de déploiement (préproduction & production)	11
6	Stratégie de tests, résilience & recettage	12
7	Organisation & qualité du code	13
8	Planification : backlog, planning poker & sprints	14
9	Roadmap technique V1 → V3 & dépendances	16
10	Budgétisation du volet développement	17
11	Conformité & numérique responsable	18
12	Zones d'incertitude & préparation du Jalon 3	19

PÉRIMÈTRE & ARTICULATION AVEC LE BUSINESS PLAN

Le **business plan collectif** porte la direction produit, le marché, le modèle économique et le financement. Ce document est la **contribution du lead technique (DFS)** attendue au Jalon 2 : choix & justification de la solution technique, infrastructure de déploiement, stratégie de tests, planification (user stories, sprints, planning poker, backlog), budgétisation et roadmap du volet développement. Tous les chiffres produit sont alignés sur le business plan Lumia.

Cadrage & analyse d'écart V0 → V1

D'un MVP en production vers une V1 commercialisable, fiable et conforme.

1.1 — Point de départ : un MVP (V0) en production

Le MVP de Lumia est **en ligne en production depuis février 2026** (vocal-assistant-two.vercel.app), construit en six semaines puis ajusté après les premiers retours terrain. Il valide la promesse du produit : un assistant de vie qui *écoute, retient et anticipe*. Le socle technique est déjà opérationnel : reconnaissance vocale en français (Web Speech API), interprétation par **Gemini 2.5 Flash** en *function calling*, création d'événements à la voix, planification automatique dans les créneaux libres, calendrier en quatre vues et persistance **Supabase**. À ce stade, aucun abonné payant : le produit est présenté au jalon de validation, avant commercialisation réelle.

1.2 — La proactivité à deux niveaux : le cœur technique

- **Niveau 1 — détection des besoins exprimés** dans la conversation vocale (déjà amorcé dans le MVP via le *function calling*).
- **Niveau 2 — apprentissage des cycles de consommation** : le **module courses automatique** détecte les intentions d'achat, mémorise les achats, estime les intervalles de réachat et déclenche une notification *avant* que l'utilisateur n'y pense. C'est le chantier différenciant de la V1.

Fév. 2026

MVP V0 EN
PRODUCTION

~8 000

LIGNES DE CODE · ~56
FICHIERS

16

USER STORIES · 4
EPICS

4,00 €

COÛT VARIABLE /
UTILISATEUR ACTIF

1.3 — Analyse d'écart V0 → V1

Le cadrage part de l'existant. Cet inventaire distingue ce qui est déjà en production de ce qui reste à construire pour la V1 :

CAPACITÉ	ÉTAT V0 (MVP ACTUEL)	CIBLE V1
Reconnaissance vocale fr-FR	Présent	+ onboarding vocal guidé en 2 min
Interprétation IA (function calling)	Présent	Couche LLM agnostique, suivi des tokens
Calendrier 4 vues & planification créneaux libres	Présent	Maintien & consolidation
Module courses automatique (niveau 2)	Absent	Détection + apprentissage + rappel push
Authentification & RLS effective	Partiel (mono-utilisateur demo-user)	Supabase email + Google OAuth, RLS sur <code>auth.uid()</code>
PWA & notifications push	Absent	Installable + Web Push
Persistance & résilience	Présent (Supabase + repli local)	Maintien
Tests automatisés & CI/CD	Partiel	Vitest + Playwright (> 70 % cœur), GitHub Actions
Conformité RGPD / IA Act	Basique	

LIEN AVEC LA GRILLE D'ÉVALUATION DFS

Ce cadrage couvre les attendus du Jalon 2 pour la filière développement : **choix et justification de la solution technique** (ch. 3-4), **infrastructure de déploiement** (ch. 5), **stratégie de tests** (ch. 6), **planification & budgétisation de la partie dev** (ch. 8 et 10) et **roadmap** (ch. 9). Il prépare le dossier individuel « Conception, développement et déploiement d'un applicatif » du Bloc 4, évalué au Jalon 3.

Spécifications techniques

L'expression des besoins V1, traduite en exigences vérifiables.

2.1 — Exigences fonctionnelles V1

RÉF.	EXIGENCE	CRITÈRE D'ACCEPTATION
EF-01	Créer / modifier / supprimer un événement à la voix et au clavier	Action exécutée et persistée < 2 s
EF-02	Décomposer une intention en plan multi-étapes placé dans les créneaux libres	Aucun chevauchement, échéance respectée
EF-03	Détecter une intention d'achat dans la conversation (proactivité niveau 1)	Item identifié et proposé à l'enregistrement
EF-04	Apprendre les cycles de consommation et rappeler avant la date estimée (niveau 2)	Notification push 1 j avant la date estimée
EF-05	Onboarding vocal complet en moins de 2 minutes	Premier événement créé sans configuration
EF-06	Compte utilisateur (email + Google OAuth) et session persistante	Route /app protégée, connexion/déconnexion OK
EF-07	Isolation stricte des données par utilisateur	RLS : un utilisateur ne lit que ses lignes
EF-08	Application installable (PWA) avec notifications push	Installation hors store, push reçu
EF-09	Suppression du compte et de toutes les données (droit à l'effacement)	Purge complète vérifiable

2.2 — Exigences non fonctionnelles

CATÉGORIE	EXIGENCE CIBLE
Performance	Réponse IA perçue < 2,5 s ; PWA fluide sur mobile d'entrée/milieu de gamme.
Coût à l'usage	Coût variable maîtrisé à 4,00 €/utilisateur actif (Gemini 3,40 € + infra 0,60 €) ; suivi des tokens réels.
Sécurité	RLS native sur toutes les tables ; sessions par cookies <i>HTTP-only</i> ; secrets en variables d'environnement ; vérification serveur (<code>getUser()</code>).
Fiabilité / résilience	Mode dégradé si l'IA ou la base est injoignable ; mises à jour optimistes ; middleware <i>fail-closed</i> .
Conformité	RGPD et IA Act (risque limité) respectés dès la conception ; transparence et contrôle humain.
Portabilité IA	Architecture agnostique au fournisseur de LLM ; bascule vers une alternative (ex. Mistral Large) isolée dans un service.
Accessibilité	Contraste ≥ 4,5:1, focus clavier visibles, <code>prefers-reduced-motion</code> respecté.
Scalabilité	

SOURCE UNIQUE DE VÉRITÉ

Le modèle de données repose sur l'entité `CalendarEvent` (début/fin obligatoires, source `voice|manual|ai`, `meta.groupId` pour relier les étapes d'un même plan). Ce choix — tout est un événement daté — simplifie la cohérence et le raisonnement de l'IA.

Architecture technique & stack

Une architecture full-stack TypeScript, serverless et orientée time-to-market.

3.1 — Vue d'ensemble

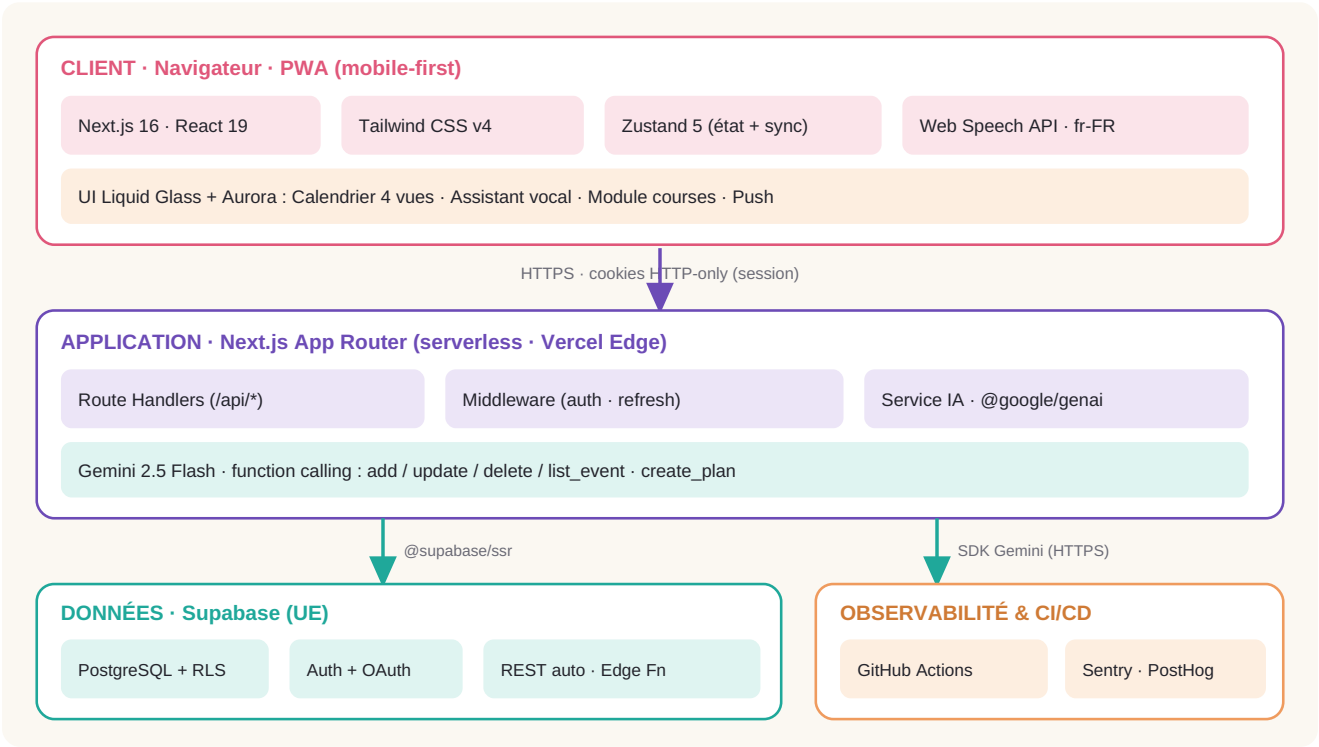


Figure 1 — Architecture cible V1 : client PWA mobile-first, application serverless Next.js sur Vercel Edge, données Supabase (UE), observabilité & CI/CD.

3.2 — Stack technologique & justification

COUCHE	CHOIX	JUSTIFICATION
Frontend	Next.js 16 (App Router)	API Routes intégrées, Server Components, déploiement Vercel optimisé ; un seul langage front + back.
UI	React 19	Concurrent Mode et hooks avancés ; composants réutilisables, itération rapide mobile-first.
Langage	TypeScript 5 (strict)	Types centralisés, réduction des bugs runtime, maintenabilité.
Styling	Tailwind CSS v4	Design tokens via variables CSS (<code>@theme</code> dans <code>globals.css</code>), palette Lumia cohérente.
État global	Zustand 5	API minimaliste ; <i>persist</i> middleware et synchronisation Supabase.
IA	Gemini 2.5 Flash	Meilleur rapport coût/latence, <i>function calling</i> natif, ≈ 33× moins cher que GPT-4o (0,075 \$/0,30 \$ par M tokens).
Base de données	Supabase (PostgreSQL)	RLS native, API REST auto-générée, hébergement UE ; time-to-market et coût maîtrisé.
Entrée vocale	Web Speech API	

Reconnaissance fr-FR native, zéro dépendance externe, aucun coût d'API.

Déploiement	Vercel Edge Network	CI/CD automatique depuis GitHub, previews par PR, rollback instantané.
-------------	---------------------	--

3.3 — Flux d'une commande vocale

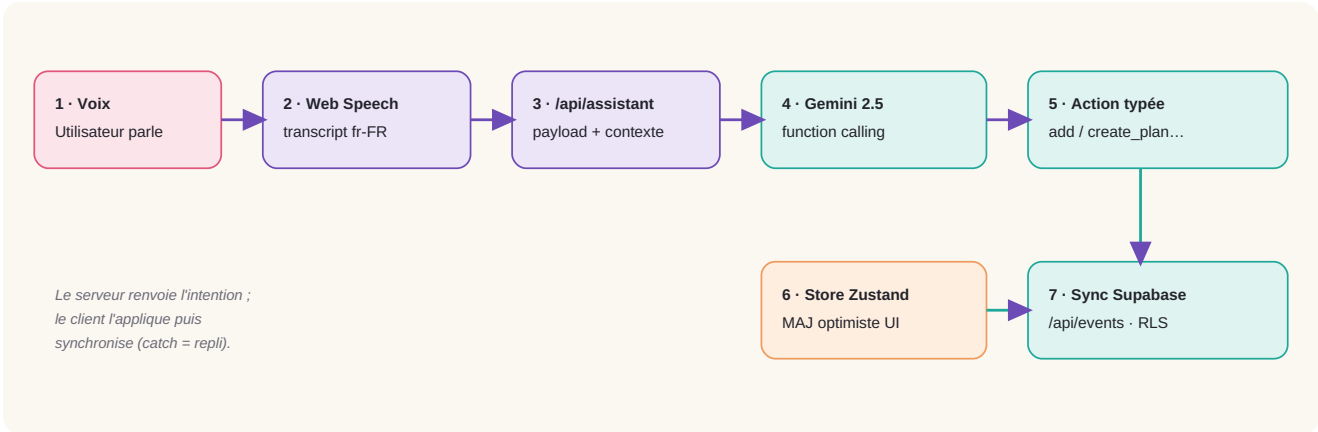


Figure 2 — Chaîne de traitement d'une commande vocale, de la parole à la persistance.

3.4 — Moteur de function calling & planification

L'API `/api/assistant` appelle le modèle `gemini-2.5-flash` avec un jeu de fonctions déclarées (*tools*). Le prompt système fournit le contexte (vue active, date de référence Europe/Paris, événements existants). Gemini renvoie un appel de fonction typé que le serveur transforme en action :

FONCTION	RÔLE
<code>add_event</code>	Crée un événement (titre, début/fin, description, lieu).
<code>update_event</code> / <code>delete_event</code>	Met à jour ou supprime un événement existant.
<code>list_events</code>	Liste les événements d'une période (jour / semaine / mois).
<code>create_plan</code>	Décompose une intention en plusieurs événements liés (<code>groupId</code>), placés dans les créneaux libres.

La planification s'appuie sur deux fonctions pures testables : `findFreeSlots()` (calcule les intervalles libres d'une période en triant les événements et en repérant les trous) et `canFitEvent()` (vérifie l'absence de chevauchement). L'IA place ainsi les étapes d'un plan dans l'ordre logique, avant l'échéance, sans conflit.

3.5 — État, synchronisation & résilience

Le store **Zustand** est la source de vérité côté client. Au démarrage, `hydrateFromSupabase()` charge les événements ; chaque mutation (`addEvent` , `addEvents` , `updateEvent` , `deleteEvent`) met l'UI à jour **de façon optimiste** puis synchronise via les route handlers `/api/events`. La résilience est intégrée : un `persist` local, un repli sur des données par défaut si Supabase est injoignable, un drapeau `isHydrated` pour éviter les doubles chargements, et la gestion explicite des erreurs IA — **429** (quota Gemini : 20 requêtes/jour en gratuit) et **503** (service surchargé) renvoient un message clair plutôt qu'un plantage. En l'absence de clé API, un mode *fallback* à base de mots-clés prend le relais.

3.6 — Décisions d'architecture (ADR)

ADR-01 · App Router Next.js plutôt qu'une SPA pure. Rendu serveur, route handlers et middleware permettent de protéger les routes et de rafraîchir la session côté serveur — indispensable pour une auth sécurisée et le référencement du site vitrine.

ADR-02 · Supabase plutôt qu'un backend sur mesure. PostgreSQL managé, Auth et RLS natifs suppriment des semaines d'infrastructure ; l'isolation des données devient une garantie au niveau base. Hébergement UE aligné RGPD.

ADR-03 · Gemini 2.5 Flash en function calling, derrière une couche agnostique. Le function calling transforme le langage naturel en actions typées et validées ; l'accès au LLM est isolé dans un service pour permettre une bascule (ex. Mistral Large) sans réécrire l'applicatif.

ADR-04 · Exécution des actions côté client après réponse de l'IA. Le serveur renvoie l'intention (fonction + arguments) ; le client l'applique au store puis synchronise. Séparation nette IA / persistance, facile à tester.

ADR-05 · @supabase/ssr + middleware plutôt qu'un JWT maison. Cookies HTTP-only, refresh automatique et vérification serveur (`getUser()`) ; surface d'attaque réduite, conformité accrue.

Expérience vocale & design system

« Liquid Glass + Aurora » : une interface apaisante, premium et voice-first.

Le produit possède un langage visuel mûr et documenté (source de vérité : `globals.css` + `design-system/MASTER.md`). Mon rôle de dev est de l'**implémenter fidèlement** et d'**unifier les deux surfaces** — l'app et le site vitrine — autour de ce système, en lien étroit avec la direction artistique (Maëlane).

4.1 — Palette & typographie

Palette (tokens)

--color-brand-*)

- Blanc #fdf8f8
- Noir texte #3d3d3d
- Rose #f4b4c8
- Saumon #f6b9ae
- Pêche #fcecd3
- Lavande #c5a6cf
- Bleu #96b6dd
- Teal #9dc0bc

Dégradé brand : 135° rose → pêche.

Texte accessible : #515151 / #636363.

Typographie & surfaces

- Police `proxima-nova` ; titres *bold, tracking-tight* ; corps *light, leading-relaxed*.
- Échelle : 12 · 14 · 16 · 18 · 24 · 32 · 48 · 60 · 72.
- Surfaces verre : `.aurora-bg`, `.glass`, `.glass-panel`, variantes `.glass-pink` / `.glass-teal`, `.glass-grain`, `.glass-highlight`.
- Rayons : `rounded-full` (CTA, pills), `rounded-2xl` (cartes).

4.2 — Expérience voice-first

Le cœur du produit est l'interaction vocale : bouton micro, visualisation d'onde, anneau de pulsation (`animate-pulse-ring`) et états explicites `idle` → `listening` → `processing` → `error` exposés par le hook `useSpeechRecognition`. Le feedback doit être immédiat et lisible, sans configuration. Le repli est gracieux quand l'API vocale n'est pas supportée par le navigateur.

4.3 — Accessibilité & sobriété (numérique responsable)

- **Contraste ≥ 4,5:1** via les variantes de texte accessibles ; **focus clavier visibles** (anneau lavande 3 px) ; navigation au clavier de la modale IA (focus trap, échap).
- **prefers-reduced-motion** respecté ; transitions douces 150-300 ms, 1 à 2 éléments animés par vue.
- **Icônes en SVG inline** (jamais d'emoji) ; anti-patterns proscrits (néon, dark mode, ombres dures, coins secs).

CHANTIER V1 : UNIFICATION APP ↔ VITRINE

L'app embarque déjà le système Liquid Glass + Aurora ; le site vitrine est resté en aplats. Un chantier de **refonte du vitrine** l'aligne sur le langage de l'app (aurora de fond, surfaces en verre, hero « show don't tell » avec aperçu produit), à charte constante, avec un **gate « build vert »** et la *pre-delivery checklist* (accessibilité, responsive 375/768/1024/1440, cohérence). Méthode en deux temps : proposition validée, puis réalisation écran par écran.

Infrastructure de déploiement

Trois environnements, un pipeline continu, un déploiement réversible.

5.1 — Environnements

ENVIRONNEMENT	PLATEFORME	RÔLE
Local (dev)	<code>npm run dev</code> · <code>.env.local</code>	Développement et tests locaux ; projet Supabase de développement.
Préproduction	Vercel Preview Deployments	Une URL éphémère générée à chaque pull request : recette, tests E2E et validation produit/design avant fusion.
Production	Vercel Edge (Pro en V1)	Déploiement automatique depuis <code>main</code> ; previews illimitées, <i>rollback</i> instantané.
Données	Supabase (projets dev & prod, UE)	PostgreSQL managé, sauvegardes, RLS ; séparation stricte dev / prod.

NOTE MÉTHODOLOGIQUE (GUIDE MDS)

Conformément au guide, l'infrastructure de préproduction mise en place par le développement est la seule infrastructure utilisée par l'équipe : pas d'infrastructure de production séparée à auditer côté cybersécurité. Les *preview deployments* Vercel jouent ce rôle de préproduction partagée.

5.2 — Intégration & déploiement continu (CI/CD)

Le pipeline **GitHub Actions** s'exécute sur chaque pull request : installation, *lint* (ESLint), vérification de types (`tsc`), tests unitaires et tests E2E Playwright. La fusion est **bloquée si le pipeline échoue**. À la fusion sur `main`, Vercel redéploie automatiquement et permet un *rollback* immédiat. Chaque déploiement rejoue donc les tests (non-régression). Le **build vert** (`npm run build` = Next.js + `tsc`) est le gate de vérification de référence.

5.3 — Observabilité & sécurité d'exploitation

- **Monitoring d'erreurs** : Sentry (front + back), alerting sur régression.
- **Analytics produit** : PostHog (entonnoirs, rétention, événements clés).
- **Secrets** : variables d'environnement Vercel / Supabase, jamais committées.
- **Quota IA** : passage à un plan Gemini payant requis pour la production (le palier gratuit est limité à 20 requêtes/jour).

Stratégie de tests, résilience & recettage

Une pyramide de tests rejouée en continu, adossée à l'expression des besoins.

6.1 — Scénarios de test par niveau

NIVEAU	PÉRIMÈTRE	OUTILS & CIBLES
Tests unitaires	Fonctions pures : <code>findFreeSlots</code> / <code>canFitEvent</code> , helpers de dates, calcul des intervalles de réachat, extraction d'action du function calling, store Zustand	Vitest · logique métier isolée
Tests d'intégration	Route handlers <code>/api/events</code> & <code>/api/conversations</code> , accès Supabase, politiques RLS, redirections du middleware, gestion 429/503	Vitest + client Supabase de test
Tests E2E	Onboarding 2 min, parcours auth (inscription / connexion / OAuth / déconnexion), création vocale d'un événement, détection d'achat → notification	Playwright (Chrome, Safari)
Tests utilisateurs	Protocole sur panel de testeurs (pilote équipe), restitution et analyse des retours	Protocole + rapport

6.2 — Tests de résilience

Au-delà du chemin nominal, des scénarios d'erreur sont explicitement testés, car la robustesse fait partie de l'expérience : indisponibilité de l'IA (**429 / 503**) avec message clair, absence de clé API (mode *fallback*), Supabase injoignable (repli sur données par défaut, conservation de la session locale), navigateur sans Web Speech API (dégradation gracieuse), middleware *fail-closed* (utilisateur traité comme déconnecté en cas de doute).

6.3 — Couverture, non-régression & recettage

- **Couverture cible > 70 %** sur le code cœur (logique métier, API, store).
- **Non-régression** : la suite est rejouée à chaque PR et à chaque déploiement ; aucune fusion sans pipeline vert.
- **Recettage final** : chaque exigence (EF-01 à EF-09) est adossée à un scénario de test ; le rapport de recette servira de preuve au dossier individuel du Jalon 3.
- **Traçabilité** : rapports d'exécution (taux de réussite, couverture) archivés à chaque sprint.

PRÉPARATION BLOC 4 (JALON 3)

Cette stratégie alimente les livrables individuels du M2 : scénarios de test, rapport d'exécution (couverture, non-régression), synthèse du recettage final, métriques de qualité de code et documentation utilisateur.

Organisation & qualité du code

Une base de ~8 000 lignes structurée en modules à responsabilité unique.

7.1 — Architecture du code

Le projet (~56 fichiers TypeScript/TSX, ~8 000 lignes) applique une séparation nette des responsabilités selon les principes SOLID et Clean Architecture :

COUCHE	RESPONSABILITÉ
types/	Types centralisés (<code>CalendarEvent</code> , actions de l'assistant, statuts vocaux).
hooks/	Logique réutilisable (<code>useSpeechRecognition</code>).
store/	État global Zustand + synchronisation Supabase.
lib/	Fonctions pures : <code>dateHelpers</code> , <code>schedulingUtils</code> , clients & accès Supabase.
services/	Appels applicatifs (<code>assistantApi</code>).
components/	UI (calendrier, chat/vocal, modale) ; sous-dossiers <code>chat/</code> & <code>calendar/</code> .
app/	Routes & route handlers (App Router) : <code>/app</code> , <code>/auth</code> , <code>/accueil</code> , <code>/api/*</code> .

7.2 — Refactoring atomique & dette technique

Un travail de **refactoring atomique** a découpé les fichiers volumineux en modules ciblés, sans changer le comportement externe : extraction des utilitaires de dates et de planification (`dateHelpers` , `schedulingUtils`), du service d'appel IA (`assistantApi`) et des sous-composants de chat (`MessageBubble` , `StatusIndicator` , `MicrophoneButton` , `SuggestionChip`) et de calendrier (`EventCard` , `DayEventsSummary`). Chaque étape est validée par un **build vert**. Cette discipline garde le code testable et prêt à accueillir les chantiers V1.

7.3 — Méthode de travail

- **Conventions** : TypeScript strict, ESLint (`eslint-config-next`), commits conventionnels, branches par fonctionnalité, revue de PR.
- **Documentation vivante** : plans & specs versionnés (`docs/`), journal de projet, `SUPABASE_SETUP.md` .
- **Développement assisté par IA** : briefs en deux temps (proposition validée puis réalisation), plans découpés en tâches vérifiables — gain de rythme pour une équipe en alternance.

Planification : backlog & sprints

16 user stories, estimées en planning poker, ordonnancées sur 6 sprints.

8.1 — Backlog initial V1

Découpage en 4 epics et 16 user stories, estimées en story points (suite de Fibonacci) et priorisées (P0 = indispensable V1, P1 = important, repli possible en V1.1).

Epic A Authentification & sécurité

ID	USER STORY	SPRINT	POINTS	PRIO
AUTH-01	Créer un compte avec email + magic link	S1	5	P0
AUTH-02	Activer la connexion Google OAuth	S1	3	P0
AUTH-03	RLS activée sur toutes les tables Supabase	S1	5	P0
AUTH-04	Supprimer son compte et toutes ses données (RGPD)	S2	8	P0

Epic B Module courses automatique (proactivité niveau 2)

ID	USER STORY	SPRINT	POINTS	PRIO
COUR-01	Détecter une intention d'achat dans la conversation (Gemini)	S2	8	P0
COUR-02	Stocker l'item acheté et la date (Supabase)	S2	3	P0
COUR-03	Calculer l'intervalle moyen de réachat après 3 achats	S3	13	P0
COUR-04	Planifier une notification push 1 j avant la date estimée	S3	8	P0
COUR-05	Apprentissage continu : ajuster l'intervalle à chaque achat	S4	5	P1

Epic C Onboarding vocal & PWA

ID	USER STORY	SPRINT	POINTS	PRIO
ONB-01	Onboarding vocal complet en 2 minutes	S4	8	P0
PWA-01	Application installable (manifest + service worker)	S4	5	P0
PWA-02	Notifications Web Push (permission, abonnement, envoi)	S5	8	P0

Epic D Conformité & observabilité

ID	USER STORY	SPRINT	POINTS	PRIO
COMP-01	Consentement données vocales + bannière cookies/ analytics	S2	3	P0
COMP-02	Politique de confidentialité & CGU + transparence IA Act	S1	3	P0
COMP-03	Monitoring Sentry + logs structurés	S5	5	P1
COMP-04	Tableau de bord KPI (PostHog)	S6	8	P1

8.2 — Planning poker & capacité

Estimations réalisées en **planning poker collectif** sur la suite de Fibonacci (1, 2, 3, 5, 8, 13, 21). La réalisation est portée par le lead technique, à un rythme contraint par l'activité salariée maintenue en parallèle par les trois associés ; la **vélocité cible** est volontairement prudente : **15 à 18 SP par sprint** de 2 semaines.

SPRINT (2 SEM.)	CONTENU	CHARGE (SP)
S1	AUTH-01, AUTH-02, AUTH-03, COMP-02	16
S2	AUTH-04, COUR-01, COUR-02, COMP-01	22
S3	COUR-03, COUR-04	21
S4	COUR-05, ONB-01, PWA-01	18
S5	PWA-02, COMP-03	13
S6	COMP-04	8
Backlog initial V1		98

Le backlog initial représente **≈ 98 story points**. Sur 6 sprints à 15-18 SP, la capacité est tendue : les stories **P1** (apprentissage continu, monitoring, tableau de bord KPI — 18 SP) constituent la variable d'ajustement et peuvent glisser en **V1.1** sans repousser la mise en ligne de la V1.

Roadmap technique V1 → V3

Trois versions, des cibles chiffrées, des dépendances inter-métiers explicites.

9.1 — Trajectoire produit

VERSION	HORIZON	CONTENU TECHNIQUE & CIBLE
V1 · Validation	2026	Chat vocal intelligent, module courses automatique, onboarding vocal 2 min, authentification Supabase (RLS), PWA & notifications push. Cible : 350 abonnés Premium (seuil de rentabilité 225, franchi au mois 9).
V2 · Croissance	2027	App native iOS & Android, agenda familial partagé, tâches ménagères & budget, Plan Famille 14,99 €, programme de parrainage. Cible : 1 000 abonnés Premium.
V3 · Expansion B2B	2028	Licences universités (marque blanche), tableau de bord RH, intégrations Teams / Slack / Workspace, budget familial avancé. Cible : ARR > 50 000 €.

9.2 — Phases & gates de la V1

ÉTAPE	REPÈRE	CONDITION
Conception & specs	M1 → M3	Spécifications V1 figées, architecture validée, recettage du MVP, backlog prêt.
Production (6 sprints)	M3 → M9	Build V1, tests continus, conformité, mise en place infra production.
Gate · Conformité	avant lancement	RGPD / IA Act vérifiés, consentement données vocales en place.
Mise en ligne V1	M9	Seuil de rentabilité (225 abonnés) visé ; suivi KPI activé.

9.3 — Dépendances inter-métiers

DÉPENDANCE	SENS	IMPACT & CONDITION
Identité de marque & maquettes UI (DA · Maëlane)	Dev ⇐ DA	L'intégration UI / l'unification du vitrine supposent charte et maquettes livrées (M1-M3). <i>Bloquant</i> pour l'habillage V1.
Priorisation & go-to-market (CdP · Antonin)	Dev ⇐ CdP	L'ordre des epics suit la valeur produit et le calendrier de lancement (M9).
Mesure des coûts d'API (tokens Gemini)	Dev ⇒ Finance	La mesure des tokens réels alimente le coût variable du business plan.
Infrastructure de préproduction	Dev ⇒ Équipe	Les preview deployments servent d'environnement de recette commun.
Conformité RGPD / IA Act	Dev ⇔ Équipe	Transverse ; à valider avant la mise en ligne (gate conformité).

Budgétisation du volet développement

Un modèle lean : main-d'œuvre internalisée, coûts d'infrastructure maîtrisés.

10.1 — Modèle de coûts & principe lean

Les trois associés conservent leur emploi salarié et **ne se versent aucune rémunération en V1**. Le coût de la main-d'œuvre n'engendre donc **aucune sortie de trésorerie** : l'effort de développement est internalisé. Cet effort est néanmoins documenté car il fonde l'éligibilité aux dispositifs non dilutifs — **crédit d'impôt innovation (CII, 20 %)** et, le cas échéant, **crédit d'impôt recherche (CIR, 30 %)**.

10.2 — Charge de développement V1 (planning, en jours-homme)

CHANTIER TECHNIQUE	DÉTAIL	CHARGE (J/H)
Architecture, specs & ADR V1	Schémas, choix techno, décisions	6
Authentification + RLS + suppression compte	Email/magic link, Google OAuth, RGPD	10
Module courses automatique	Détection, stockage, intervalles, apprentissage	18
PWA & notifications push proactives	Manifest, service worker, Web Push, scheduling	12
Onboarding vocal 2 min	Parcours d'activation guidé	6
Intégration UI, design system & refonte vitrine	Liquid Glass + Aurora, unification app ↔ vitrine	8
Tests (unitaires + E2E Playwright) & qualité	Couverture cœur, non-régression	8
CI/CD & observabilité	GitHub Actions, Sentry, PostHog	4
Conformité (vocal, CGU, IA Act, registre)	Consentement, transparence, gouvernance	4
Documentation technique & utilisateur	README, guide, doc V1	4
Total charge de développement V1		80 j/h

Charge indicative de planification (≈ 98 SP au backlog) ; non monétisée du fait du modèle sans rémunération en V1.

10.3 — Coûts d'infrastructure & outils (sorties réelles)

POSTE	DÉTAIL	MONTANT
Infrastructure M1-M3	Vercel Pro + Supabase Pro (démarrage)	150 €
Outils M1-M3	GitHub, design, monitoring	300 €
Infrastructure récurrente (lean)	Part « infra » des coûts fixes mensuels	≈ 50 €/mois
Coût variable par utilisateur actif	Gemini 3,40 € + infra Vercel/Supabase 0,60 €	4,00 €

Les plans Pro à coût fixe couvrent jusqu'à ≈ **5 000 utilisateurs** ; la marge brute unitaire est de **5,99 € (60 %)** pour un abonnement Premium à 9,99 €.

SENSIBILITÉ AU COÛT DE L'IA

Une hausse de 50 % du tarif Gemini porterait le coût variable de 3,40 € à 5,10 €/utilisateur et la marge brute de 60 % à ≈ 49 %. Atténuation : architecture *agnostique* au LLM, bascule vers Mistral Large estimée à $\approx$ **3 jours de développement, déjà prototypée**.

Conformité & numérique responsable

La conformité intégrée dès la conception, sur un terrain de données sensibles.

11.1 — RGPD

- **Bases légales** : exécution du contrat pour la gestion des événements ; **consentement explicite** pour les données vocales.
- **Minimisation & isolation** : collecte limitée au nécessaire ; isolation stricte par RLS (`user_id = auth.uid()`) ; hébergement en UE (Vercel + Supabase).
- **Droit à l'effacement** : suppression du compte et purge complète des données (US AUTH-04).
- **Transparence & gouvernance** : politique de confidentialité, CGU, bannière de consentement (COMP-01/02) ; registre des traitements documenté.
- **Enfants (V2)** : consentement parental renforcé lors de l'ouverture aux usages familiaux.

11.2 — IA Act

Lumia est classifié « **risque limité** » au sens de l'IA Act européen : les obligations principales sont la **transparence** (l'utilisateur sait qu'il interagit avec une IA) et le **maintien du contrôle humain**. L'IA n'opère aucune décision automatisée à fort impact ; une note de gouvernance du LLM (finalités d'usage de Gemini) est tenue à jour.

11.3 — Numérique responsable

- **Sobriété d'infrastructure** : architecture serverless (pas de serveur en veille permanente), hébergement UE à empreinte maîtrisée.
- **Sobriété d'usage de l'IA** : modèle Flash (faible coût/latence), suivi des tokens et optimisation des appels.
- **Accessibilité** : WCAG appliqué aux parcours clés (cf. design system, ch. 4).

COUVERTURE DES ATTENDUS DFS

Le respect de la réglementation des données (RGPD, IA Act) et l'accessibilité font explicitement partie des compétences évaluées de la filière ; ils sont intégrés au cadrage, au design system, au backlog et à la budgétisation — non traités en fin de projet.

Zones d'incertitude & préparation du Jalon 3

Les risques techniques identifiés sont assortis de pistes de traitement.

ZONE D'INCERTITUDE	CRITICITÉ	PISTE DE TRAITEMENT (JALON 3)
Dépendance & coût de l'API Gemini (non plafonné à l'échelle)	Forte	Architecture agnostique, bascule Mistral Large (≈ 3 j, prototypée) ; mesure des tokens réels ; cache et optimisation.
Quota gratuit Gemini (20 req/jour) insuffisant en prod	Modérée	Passage à un plan payant dès la préproduction ; gestion 429 déjà en place.
Module courses : qualité de la détection & de l'apprentissage	Modérée	Prompts & règles éprouvés sur jeux de tests ; ajustement continu de l'intervalle.
Support Web Push sur iOS / Safari	Modérée	Repli notification e-mail / in-app ; spike technique en début de Phase 2.
Capacité de réalisation (équipe en activité parallèle)	Modérée	Vélocité prudente, P1 reportables en V1.1, périmètre V1 resserré.

12.1 — Ce que ce cadrage prépare pour le Jalon 3

Le dossier individuel du Bloc 4 (M2) reprendra et approfondira ce cadrage : conception détaillée (specs, architecture, infrastructure), développement et déploiement de l'appliquatif, stratégie et rapport de tests (couverture, non-régression, métriques de qualité de code), recettage final, documentation utilisateur et lien vers le code source. La préproduction décrite ici fournit le socle méthodologique et budgété pour exécuter cette réalisation sans dérive.

CONCLUSION

Le volet développement du Jalon 2 transforme un MVP en production en un plan de production crédible : une architecture justifiée et portable, un moteur de function calling et de planification déjà éprouvé, un design system « Liquid Glass + Aurora » documenté, une infrastructure de déploiement continue, une stratégie de tests adossée aux besoins, un backlog de 16 user stories estimé en planning poker et ordonnancé sur 6 sprints, et un modèle de coûts lean cohérent avec le business plan. L'équipe vise une **mise en ligne de la V1 en 2026**, avec un seuil de rentabilité atteignable à 225 abonnés (visé au mois 9).